

BMS INSTITUTE OF TECHNOLOGY AND MANAGEMENT

Autonomous Institute under VTU, Belagavi, Karnataka - 590 018

Yelahanka, Bengaluru, Karnataka - 560 119



Information and Network Security (BCS701) CCA Report

On

**Codecov Bash Uploader Supply-Chain Attack (2021): Analysis and SignGate, a
Cryptographic Verify-Before-Execute Solution**

BACHELOR OF ENGINEERING

in

INFORMATION SCIENCE AND ENGINEERING

by

NAME

USN

Under the Guidance of

Prof. Sonnegowda K

Assistant Professor

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

Avalahalli, Yelahanka, Bengaluru, Karnataka -560119

2025-26

Evaluation Sheet

Name	USN	Problem Understanding & Requirements	Design & Implementation	Security Analysis Testing	Innovation & Application Relevance	Documentation & Presentation	Total
		3	9	2	2	4	20

Content	Page No.
1. Introduction.....	1
2. Abstract.....	3
3. Problem Statement.....	4
4. System Requirements.....	5
5. Design and Implementation.....	6
6. Security Analysis.....	12
7. System Testing.....	14
8. Innovation and Application Relevance.....	17
9. Conclusion.....	19
10. References.....	20

1. INTRODUCTION

1.1 Organisation and Sector

Codecov is a Software-as-a-Service (SaaS) code-coverage platform used by development teams across the software industry. After the automated tests of a project run inside a Continuous Integration / Continuous Delivery (CI/CD) pipeline, a small program called the **Codecov uploader** collects the coverage report and sends it to Codecov. In 2021 the most common way to run it was the **Bash Uploader**: a shell script downloaded and executed on every build with the one-liner `bash <(curl -s https://codecov.io/bash)`. Codecov reported more than 29,000 customers at that time [3], including large technology and security companies.

The uploader runs **inside** the customer's CI job. That environment normally holds the organisation's most sensitive credentials: cloud access keys, GitHub/GitLab tokens, package-registry tokens, database URLs and sometimes release-signing keys. So the uploader is a highly trusted piece of third-party code. Anyone who can change it can read all of these secrets.

1.2 Incident Context

On **31 January 2021** an attacker began making unauthorised, periodic changes to the Bash Uploader stored in Codecov's Google Cloud Storage (GCS) bucket [1]. The modified script sent the full list of environment variables and the git remote URLs of every CI job that ran it to a server controlled by the attacker. The compromise went unnoticed for about two months, until **1 April 2021**. That day a customer calculated the SHA-256 of the downloaded script, found it did not match the value Codecov published on GitHub, and reported it [1]. Codecov disclosed the incident publicly on **15 April 2021** [1]. Its post-mortem traced the root cause to an **HMAC key for a GCS service account left in an intermediate layer of Codecov's public Self-Hosted Docker image** [2]. Downstream victims such as HashiCorp had to rotate secrets, including a GPG key used to sign software releases [4], and investigators reported that hundreds of customer networks had been accessed [3].

1.3 Security Requirements Violated

The incident is a classic **software supply-chain attack**. The attacker did not break into each victim. They changed one trusted component that thousands of victims ran automatically. Table 1 maps the incident to the security requirements it violated.

Table 1: Security requirements violated in the Codecov incident

Requirement	How it was violated	Severity
Integrity (primary)	The uploader customers downloaded was not the one Codecov released. It had an extra exfiltration line.	Critical
Authentication (of origin)	Customers could not verify who produced the script. Anyone with bucket write access was effectively 'Codecov'.	Critical
Confidentiality	Environment variables (tokens, keys, credentials) and private repository URLs were leaked to the attacker.	High
Access control / key management	A long-lived storage HMAC key was left in a public image layer and had write permission on the distribution bucket.	High
Non-repudiation / accountability	Nothing recorded which releases were legitimate, so the tampering could not be detected or attributed for about 2 months.	Medium

This report analyses the attack and then designs and implements **SignGate**, a cryptographic 'verify-before-execute' gate. It uses digital signatures, SHA-256 hashing, a small public-key infrastructure with revocation, a hash-chained transparency log and least-privilege execution, so that a tampered uploader is never trusted or run.

2. ABSTRACT

Software supply-chain attacks compromise a trusted vendor component instead of the final target, so one breach spreads to thousands of organisations. This report studies the 2021 **Codecov Bash Uploader compromise**. An HMAC storage key leaked through a Docker image layer let an attacker quietly modify a script that customers ran in their CI/CD pipelines. For about two months it exfiltrated credentials from those pipelines. The analysis identifies six weaknesses: a secret left in a build artifact, a static over-privileged storage key, no code signing, the 'curl | bash' execution pattern, CI jobs exposing every secret to every tool, and no integrity monitoring.

As a solution we design and implement **SignGate**, a lightweight verify-before-execute gate for CI tools. Every release is hashed with **SHA-256** and its manifest is signed with **Ed25519**, using a release key that is certified by an offline root key (a mini-PKI with 90-day key rotation and a signed revocation list). Every release is also recorded in an append-only **hash-chained transparency log**. Before running the uploader, the CI pipeline checks six things in order: key certificate, revocation status, signature, hash integrity, anti-rollback and log inclusion. If any check fails, the uploader does not run (fail-closed). When it does run, it sees only an allow-listed set of environment variables. A publisher-side monitor re-verifies the served file every few minutes.

A Python prototype (about 500 lines, using the pyca/cryptography library) was tested against seven attack scenarios, from the exact 2021 attack to a stolen signing key and a version rollback. **All malicious variants were blocked** and the legitimate release was accepted. Verification took about **0.6 ms** per pipeline run. In an assume-breach test, the secrets readable by a malicious uploader fell from **8 of 8 to 1 of 8**.

Keywords: software supply chain, code signing, Ed25519, SHA-256, PKI, key revocation, transparency log, CI/CD security, least privilege.

3. PROBLEM STATEMENT

CI/CD pipelines download third-party tools and execute them with full access to production secrets, but they have **no enforced way to confirm that a tool is authentic and unmodified** before running it. In the Codecov case, one leaked storage credential was enough to turn a trusted uploader into a credential stealer for thousands of pipelines. Nobody noticed for about two months.

3.1 Threats and Vulnerabilities Identified

Table 2: Vulnerabilities exploited in the Codecov attack

ID	Vulnerability	Exploited by attacker as
V1	Secret (GCS HMAC key) left in an intermediate layer of a public Docker image	Initial access: key extracted from image layers [2]
V2	Static, long-lived storage key with write access to the release bucket	Privilege: overwrite the Bash Uploader at will
V3	No digital signature on the uploader. The SHA-256 was published but not signed, and few integrations checked it [1], [2]	Tampering went unchecked by clients
V4	'curl bash' pattern executes whatever the server returns	Execution inside every customer CI job
V5	Every step of a CI job can read every secret in the environment	Mass exfiltration of unrelated credentials (cloud, Git, GPG)
V6	No monitoring of the served artifact against the released one	About 2 months of dwell time (31 Jan to 1 Apr 2021)

3.2 Objectives

- **O1:** Guarantee **integrity**: a modified uploader must be detected before execution.
- **O2:** Guarantee **authenticity**: only software signed by the genuine publisher key is accepted, even if the attacker controls the storage bucket and the published hash.
- **O3:** Provide **key management**: protected keys, periodic rotation and fast revocation of a leaked key without changing any customer pipeline.
- **O4:** Detect **silent misuse** of a stolen key and **rollback** to old vulnerable versions.
- **O5:** **Limit the blast radius** if malicious code ever runs (least privilege).
- **O6:** Keep it **simple and cheap**: millisecond overhead, standard algorithms, easy CI integration.

4. SYSTEM REQUIREMENTS

4.1 Functional Requirements

Table 3: Functional requirements

ID	Requirement
FR1	Publisher can generate keys, sign a release manifest and publish it with the artifact.
FR2	Root key can certify release keys (with validity period) and issue a signed revocation list.
FR3	Every release is appended to a hash-chained transparency log with a signed head.
FR4	CI gate verifies certificate, revocation, signature, hash, version and log inclusion, then executes the tool only if all checks pass.
FR5	Tool is executed with an environment-variable allowlist defined in a policy file.
FR6	Monitor compares the served artifact and log entries with known releases and raises alerts.

4.2 Security (Non-Functional) Requirements

Table 4: Security requirements mapped to objectives

ID	Requirement	Property
SR1	Any single-bit change to the artifact is detected (SHA-256)	Integrity
SR2	Signatures cannot be forged without the private key (Ed25519, about 128-bit security)	Authenticity
SR3	Trust anchor (root public key) is pinned in the customer repo, never fetched from the bucket	Authentication
SR4	Private keys never stored in plain text: HSM/KMS in production, AES-256 encrypted PKCS#8 in demo	Key confidentiality
SR5	Gate is fail-closed: any error means no execution	Integrity / availability trade-off
SR6	Verification overhead under 50 ms per pipeline run	Performance

4.3 Hardware and Software Requirements

Table 5: Hardware / software environment

Component	Prototype (this project)	Production deployment
Hardware	Any x86-64 / ARM machine, 2 GB RAM	Cloud CI runners, HSM or Cloud KMS
Language / libs	Python 3.9+, pyca/cryptography 41+, Bash	Same, packaged as a CI action
Key storage	AES-256 encrypted PKCS#8 files	AWS KMS / GCP Cloud KMS / YubiHSM (root offline)
Distribution	Local folder simulating a GCS bucket	GCS / CDN over TLS 1.3, workload identity
CI platform	Any shell (demo), GitHub Actions example	GitHub Actions, GitLab CI, CircleCI

5. DESIGN AND IMPLEMENTATION

5.1 Incident Description: Timeline and Attack Vector

Figure 1 shows the attack chain. Figure 2 shows the timeline reconstructed from Codecov's security update [1], post-mortem [2] and news reports [3], [5].

How the Codecov Bash Uploader was compromised (Jan to Apr 2021)

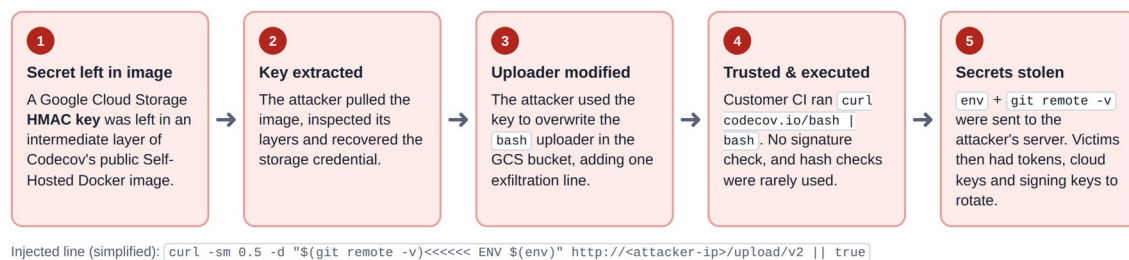


Figure 1: Attack chain of the Codecov Bash Uploader compromise

Incident timeline

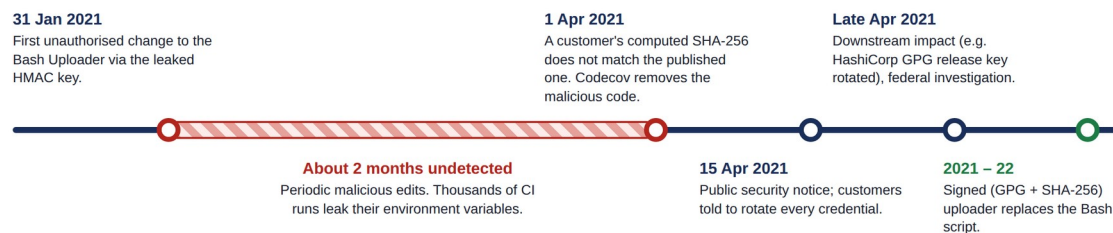


Figure 2: Incident timeline (Jan to Apr 2021 and remediation)

Attack vector. (1) Codecov's public Self-Hosted Docker image contained, in an intermediate layer, an HMAC key for a GCS service account. Deleting a file in a later layer does not remove it from earlier layers. (2) The attacker extracted the key and (3) used it to overwrite the Bash Uploader in the bucket, adding one line that posted `$(env)` and `git remote -v` to an external IP address [1], [6]. (4) Customer pipelines downloaded and executed the script with no signature check. (5) Secrets were stolen and later used against downstream organisations [3], [4]. The published SHA-256 was the only integrity control. Because it was not signed, not enforced and rarely checked, it detected the attack only when one customer happened to compare it manually.

5.2 Design Overview: SignGate

The key idea is to **move trust from the storage location to cryptographic keys**. In 2021, 'the file is in Codecov's bucket' was treated as proof that the file was genuine. SignGate

assumes an attacker **can** write to the bucket, and still makes the tampered file useless, because only the holder of the release key can produce a valid signature. The same design also handles a stolen release key. The architecture has three zones (Figure 3).

Proposed architecture: SignGate verify-before-execute pipeline

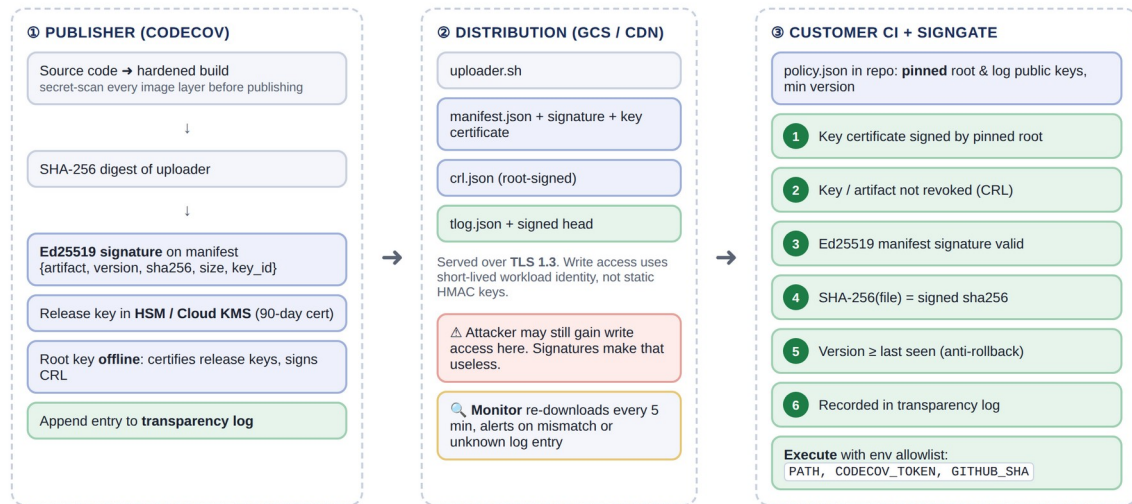


Figure 3: Proposed SignGate architecture: publisher, distribution and customer CI zones

- **Publisher (Codecov):** hardened build (secret-scan every image layer, fixing V1). SHA-256 digest, Ed25519-signed manifest, release key held in HSM/KMS, offline root key, transparency-log entry.
- **Distribution (GCS/CDN):** serves the artifact, signed manifest, root-signed revocation list and log over TLS 1.3. Writes use short-lived workload identity instead of static HMAC keys (fixing V2). A monitor re-downloads the artifact every few minutes (fixing V6).
- **Customer CI + SignGate:** the pinned policy file holds the root and log public keys. The gate runs six checks and then executes with an environment allowlist (fixing V3, V4, V5).

5.3 Cryptographic Building Blocks

Table 6: Algorithms and protocols used and why they fit

Mechanism	Standard	Role in SignGate	Why chosen
SHA-256	FIPS 180-4 [8]	Artifact digest, log hash chain, key IDs	Collision resistant; a 1-bit change gives a completely different digest
Ed25519 signature	RFC 8032 [7], FIPS 186-5 [9]	Signs manifests, key certificates, CRL, log head	~128-bit security, 64-byte signatures, fast, deterministic (no nonce-reuse bugs as in ECDSA)

Mini-PKI (root → release key)	X.509-style chain, NIST SP 800-57 [10]	Root certifies 90-day release keys	Leaked key can be revoked or rotated without customer changes
Signed revocation list	Similar to X.509 CRL	Blocks stolen keys / bad artifacts	Fast incident response
Hash-chained transparency log	Idea from Certificate Transparency RFC 6962 [11], Sigstore [12]	Public, append-only record of releases	Makes silent use of a stolen key visible
AES-256 (PKCS#8)	FIPS 197	Encrypts private keys at rest (demo)	Stands in for HSM protection
TLS 1.3	RFC 8446 [16]	Transport of the bundle	Confidentiality and server authentication in transit
HMAC key → workload identity	OIDC short-lived tokens	Bucket write access	Removes the long-lived secret that caused the breach

Why a signature and not just a hash? A hash proves 'the file matches the hash'. But an attacker who can change the file can usually change the published hash too. A digital signature ties the hash to a private key the attacker does not have. Verification uses only the public key, which customers pin in their own repository. A signature therefore provides integrity, authenticity and non-repudiation. A hash alone provides only integrity against accidental change.

5.4 Signed Release Manifest

For every release the publisher builds a manifest containing the artifact name, version, SHA-256, size, release time and key ID. The manifest is serialised as canonical JSON (sorted keys) and signed with the release key: $\sigma = \text{Sign}(\text{SK_release}, \text{canonical}(\text{manifest}))$. The key certificate is attached so that the verifier can build the chain root → release key → manifest.

```
{ "body": { "artifact": "codecov-uploader.sh", "version": "1.4.0",
            "sha256": "ea1d27b7a2f1f4f2d98d5d57c4c02cd6ae59e7ab...",
            "size": 347, "released_at": 1790000000, "key_id": "a8f83acbbe9c57a8"
          },
  "signature": "base64(Ed25519 signature over canonical body)",
  "key_certificate": { "body": { "key_id", "public_key", "purpose",
                                "not_before", "not_after" },
                      "signature": "base64(root signature)" } }
```

5.5 Key Management

Key management is where Codecov actually failed: a key sat in a public image. SignGate uses a two-level hierarchy with separation of duties (Figure 4), following NIST SP 800-57 [10].

Key management: hierarchy and lifecycle

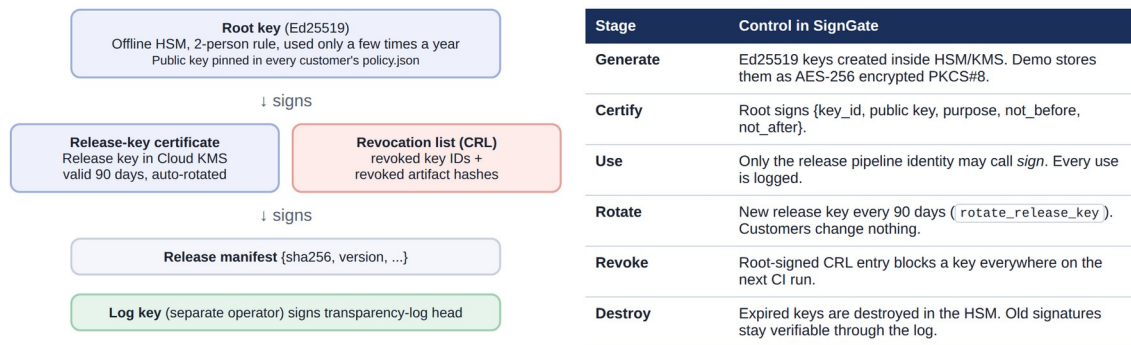


Figure 4: Key hierarchy and key lifecycle

- **Root key:** generated and kept offline in an HSM under a two-person rule. It is used only to certify release keys and sign the revocation list. Its public key is the **trust anchor** pinned in every customer's policy.json.
- **Release key:** kept in Cloud KMS. Only the release pipeline identity may call 'sign'. Its certificate is valid for 90 days and it is rotated automatically (`rotate_release_key()`).
- **Log key:** held by a separate log operator, who signs the log head, so one stolen key cannot both sign and log a release.
- **Revocation:** if a key leaks, the root signs a new CRL. Every CI gate rejects the key on its next run. This is how HashiCorp-style key exposures [4] should be contained.

5.6 Transparency Log

Every legitimate release is appended to a public, append-only log. Each entry stores $h_i = \text{SHA-256}(h_{(i-1)} \parallel \text{SHA-256}(\text{manifest}_i))$ with $h_0 = 0^{256}$, so editing or deleting any old entry changes every later hash. The log operator signs the head (size, h_n). The gate accepts a release only if its manifest is in the log, the chain recomputes to the signed head, and the log has not shrunk since the last run. So an attacker with a stolen release key has two options. They can skip the log, and every gate rejects the release. Or they can log it, and the publisher's monitor sees a release it never made and revokes the key.

5.7 Verify-Before-Execute Gate

Figure 5 shows the decision flow. The order matters. The cheapest checks that do not depend on the artifact run first, and every failure stops execution immediately (fail-closed).

SignGate decision flow (fail-closed)

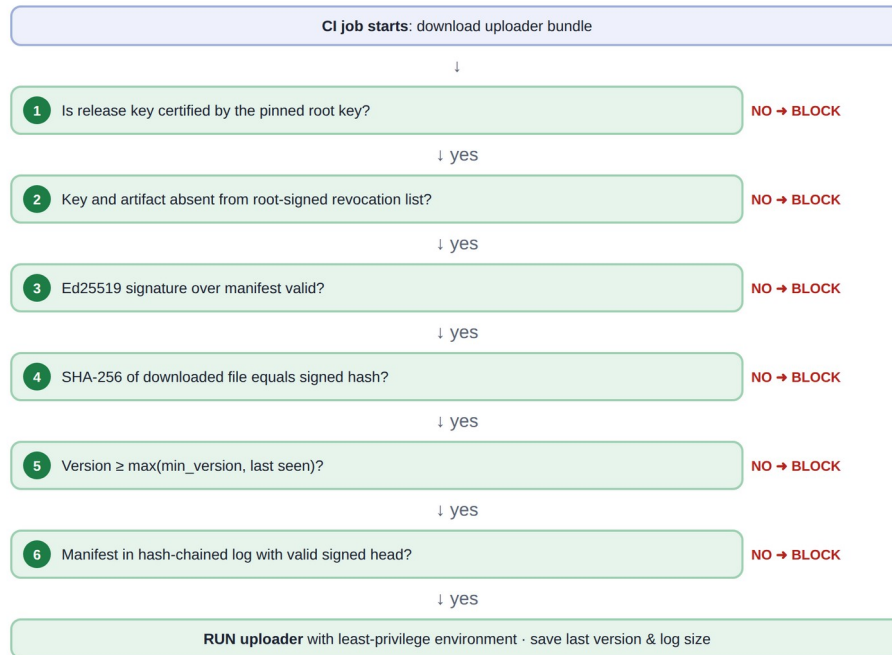


Figure 5: SignGate decision flow executed in the CI job

After all six checks pass, the gate stores the accepted version and log size (state.json) for anti-rollback. It then runs the uploader with **only the allow-listed environment variables** (e.g. PATH, CODECOV_TOKEN, GITHUB_SHA). Cloud keys, Git tokens and signing keys are simply not visible to the tool, which fixes V5.

5.8 Implementation

The prototype is written in Python 3 using the audited pyca/cryptography library for Ed25519 and key serialisation. Table 7 lists the modules. The 2021 Bash Uploader is simulated with a small shell script, and the malicious line is reproduced, but it writes the stolen variables to a local file instead of sending them to a server.

Table 7: Implementation modules

Module	Responsibility
crypto_utils.py	SHA-256, canonical JSON, Ed25519 key generation / sign / verify, AES-256 encrypted PKCS#8 key storage
pki.py	Root-signed release-key certificates (validity, purpose) and revocation list
tlog.py	Hash-chained append-only log, signed head, chain and inclusion verification
publisher.py	Key ceremony, release (hash → sign → log → publish), rotate, revoke
gate.py	The six-check verifier, env allowlist, fail-closed execution
monitor.py	Publisher-side tamper and unknown-release detection
cli.py	Commands: init, release, run, revoke, monitor

demo/attack_demo.py	Replays the 2021 attack and five stronger variants; measures timing
tests/test_signgate.py	13 automated unit tests

Core of the gate (simplified from gate.py):

```
ok, why, release_pub = pki.check_key_certificate(manifest['key_certificate'],
ROOT_PUB)
ok, why = pki.check_revocation(crl, ROOT_PUB, body['key_id'], body['sha256'])
ok = crypto.verify(release_pub, body, manifest['signature'])          # Ed25519
ok = crypto.sha256_file(artifact) == body['sha256']                  # integrity
ok = version(body['version']) >= max(policy.min_version, state.last_version)
ok, why = check_log(log, signed_head, LOG_PUB, body, state.last_log_size)
subprocess.run(['bash', artifact], env=minimal_env(policy.env_allowlist))
```

CI integration. The vulnerable one-liner bash `<(curl -s https://codecov.io/bash)` is replaced by: `python -m signgate.cli run --bucket bucket/ --policy .signgate/policy.json`. The policy file, which contains the pinned public keys, is committed in the customer's repository and never downloaded from the same bucket as the uploader (demo/ci-example.yml).

6. SECURITY ANALYSIS

6.1 Resistance to the Original Attack Vector

In the real attack the adversary had **write access to the bucket** but **not** to any signing key. Under SignGate the modified script produces a different SHA-256 (second-preimage resistance of SHA-256), so check 4 fails. If the attacker also rewrites the hash in the manifest, the Ed25519 signature no longer verifies (check 3). Forging it requires the release private key, which is infeasible (about 2^{128} operations). The attacker cannot substitute their own key either, because the certificate must chain to the root key pinned in the customer's repository (check 1). **The exact 2021 attack is therefore blocked before a single line of the script runs.** The monitor would also have raised an alert within minutes of 31 January 2021, instead of about 2 months later.

6.2 Threat Model (STRIDE)

Table 8: STRIDE threat model for the uploader supply chain

STRIDE	Threat	SignGate control	Check
Spoofing	Attacker publishes uploader as 'Codecov' with own key	Key certificate must chain to pinned root	1
Tampering	Uploader modified in bucket (2021 attack)	SHA-256 bound by Ed25519 signature	3, 4
Tampering	Old signed version replayed (rollback)	Signed version $\geq$ last seen	5
Repudiation	Publisher denies / attacker hides a release	Signed manifests + append-only log	6
Information disclosure	Malicious code reads CI secrets	Env allowlist (least privilege)	exec
Denial of service	Attacker corrupts files to break builds	Fail-closed (safe); mirror copies restore service	-
Elevation of privilege	Stolen release key used to sign malware	Transparency log + monitor + CRL revocation	2, 6

6.3 Attacker Capability Analysis

Table 9: What each attacker capability achieves

Attacker controls	Legacy Codecov (2021)	With SignGate
Bucket write only (actual 2021 case)	Full compromise of all pipelines	Blocked (check 4 / 3)
Bucket + published hash	Full compromise	Blocked (check 3)
Bucket + own signing key	Full compromise	Blocked (check 1)
Stolen release key	Full compromise	Blocked unless logged; if logged $\rightarrow$ alert $\rightarrow$ revoked
Stolen release key + log key	Full compromise	Monitor detects unknown entry $\rightarrow$ CRL revocation

Offline root key	n/a	Out of scope: protected by HSM + two-person rule
------------------	-----	--

6.4 Limitations and Residual Risk

- If the **publisher's own build system** is compromised before signing (as in SolarWinds), a malicious build is signed legitimately. SignGate makes it visible in the log but cannot judge intent. Reproducible builds and SLSA provenance [13] address this.
- The **root key** is the single point of trust. It is mitigated by offline HSM storage, a two-person rule and rare use.
- Customers must **adopt** the gate. Codecov's own post-incident signed uploader failed partly because verification was optional [2]. SignGate therefore makes verification the default execution path.
- Fail-closed can cause build failures if the distribution service is unavailable. This is an accepted trade-off of availability for integrity.

7. SYSTEM TESTING

7.1 Test Approach

Testing combines **attack simulation** (demo/attack_demo.py) with **unit tests** (tests/test_signgate.py). Each scenario creates a fresh publisher, bucket and customer pipeline in a temporary folder, lets the attacker change something, and then runs the real gate code. A test passes when the gate's decision equals the expected decision.

Table 10: Attack-simulation test cases and results

ID	Scenario	Expected	Actual	Stopped at
S1	Legitimate signed release	ALLOW	ALLOWED	-
S2	Codecov 2021 attack: uploader edited in bucket	BLOCK	BLOCKED	4. SHA-256 integrity
S3	Attacker also rewrites published SHA-256	BLOCK	BLOCKED	3. Ed25519 signature
S4	Attacker signs with own key + fake cert	BLOCK	BLOCKED	1. Key certificate (PKI)
S5	Stolen release key, release not logged	BLOCK	BLOCKED	6. Transparency log
S6	Stolen key detected by monitor and revoked	BLOCK	BLOCKED	2. Revocation list
S7	Rollback to older signed version	BLOCK	BLOCKED	5. Anti-rollback

7.2 Results

```
$ python demo/attack_demo.py
=====
SignGate attack simulation - Codecov Bash Uploader supply-chain compromise
=====

S1: Legitimate signed release
[PASS] 1. Key certificate (PKI)      release key certified by root
[PASS] 2. Revocation list           key and artifact not revoked
[PASS] 3. Ed25519 signature         manifest signature valid
[PASS] 4. SHA-256 integrity         artifact hash matches signed manifest
[PASS] 5. Anti-rollback             version 1.4.0 is current
[PASS] 6. Transparency log          release found in append-only log, chain intact
=> ALLOWED (expected ALLOW) OK

S2: Codecov 2021 attack: uploader edited in bucket
[PASS] 1. Key certificate (PKI)      release key certified by root
[PASS] 2. Revocation list           key and artifact not revoked
[PASS] 3. Ed25519 signature         manifest signature valid
[FAIL] 4. SHA-256 integrity          HASH MISMATCH: expected eald27b7a2f1.. got 06b7c8163077..
=> BLOCKED (expected BLOCK) OK

S3: Attacker also rewrites published SHA-256
[PASS] 1. Key certificate (PKI)      release key certified by root
[PASS] 2. Revocation list           key and artifact not revoked
[FAIL] 3. Ed25519 signature          manifest signature INVALID (forged/edited)
=> BLOCKED (expected BLOCK) OK

S4: Attacker signs with own key + fake cert
[FAIL] 1. Key certificate (PKI)      key certificate NOT signed by pinned root key
=> BLOCKED (expected BLOCK) OK
```

Figure 6: Test output: legitimate release allowed; 2021 attack and forgery variants blocked

```

... (continued)
S5: Stolen release key, release not logged
[PASS] 1. Key certificate (PKI)      release key certified by root
[PASS] 2. Revocation list           key and artifact not revoked
[PASS] 3. Ed25519 signature         manifest signature valid
[PASS] 4. SHA-256 integrity         artifact hash matches signed manifest
[PASS] 5. Anti-rollback             version 1.4.1 is current
[FAIL] 6. Transparency log          release NOT recorded in transparency log
=> BLOCKED (expected BLOCK) OK

S6: Stolen key detected by monitor and revoked
[PASS] 1. Key certificate (PKI)      release key certified by root
[FAIL] 2. Revocation list           release key a77a410d46088aae has been REVOKED
=> BLOCKED (expected BLOCK) OK
Monitor: ALERT: unknown release v1.4.1 in log (index 1) - possible key theft

S7: Rollback to older signed version
[PASS] 1. Key certificate (PKI)      release key certified by root
[PASS] 2. Revocation list           key and artifact not revoked
[PASS] 3. Ed25519 signature         manifest signature valid
[PASS] 4. SHA-256 integrity         artifact hash matches signed manifest
[FAIL] 5. Anti-rollback             version 1.3.0 older than 1.4.0 (downgrade)
=> BLOCKED (expected BLOCK) OK

S8: Assume-breach - malicious uploader executes anyway
Legacy 'curl | bash' : 8/8 secrets exposed ->
    CODECOV_TOKEN, AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, GITHUB_TOKEN,
    NPM_TOKEN, DOCKER_PASSWORD, GPG_SIGNING_KEY, DATABASE_URL
SignGate env allowlist: 1/8 secrets exposed -> CODECOV_TOKEN

S9: Publisher integrity monitor
Before tamper: no alerts
After tamper : ['ALERT: served uploader 06b7c8163077.. differs from signed release eald27b7a2f1..']

Average verification time: 0.56 ms per pipeline run

Summary: 7/7 scenarios behaved as expected

```

Figure 7: Test output: stolen key, revocation, rollback, assume-breach, monitor and timing

Assume-breach test (S8). To test defence-in-depth, the malicious uploader was executed directly, as if every check had been bypassed. Eight typical CI secrets were placed in the environment. With the legacy method all 8 were captured. With SignGate's allowlist only the low-value, upload-only CODECOV_TOKEN was visible (Figure 8).

Assume-breach test: CI secrets readable by a malicious uploader

Legacy `curl | bash` : full CI environment

8 / 8 secrets exposed (AWS keys, GitHub token, NPM, Docker, GPG key, DB URL, Codecov token)

SignGate: environment allowlist

1 / 8 only CODECOV_TOKEN (low-value, upload-only token)

87.5 % fewer secrets exposed, even in the worst case where a malicious uploader somehow executes.

Figure 8: Secrets exposed to a malicious uploader: legacy versus SignGate

Monitor test (S9). The monitor reported no alerts on the genuine bucket. After a one-line change it immediately reported that the served uploader differs from the signed release.

Performance. The full six-check verification averaged **0.56 ms** over 200 runs on a standard CI container, far below SR6 (50 ms). **Unit tests:** 13 of 13 passed, covering signature round-

trip, one-byte hash change, wrong passphrase on encrypted keys, log history edits and all attack scenarios.

Table 11: Requirement validation summary

Requirement	Validated by	Status
SR1 Integrity	S2 blocked on hash mismatch; unit test one-byte change	Met
SR2 Authenticity	S3, S4 blocked on signature / certificate	Met
SR3 Pinned trust anchor	S4: attacker's self-signed cert rejected	Met
SR4 Key protection	Unit test: encrypted key unusable without passphrase	Met (demo level)
SR5 Fail-closed	Every failing check stopped execution (S2–S7)	Met
SR6 Performance	0.56 ms average verification	Met

8. INNOVATION AND APPLICATION RELEVANCE

8.1 What is New Compared to the Standard Fix

The standard fix, and what Codecov did after the incident, was to publish a GPG-signed uploader with a SHA-256 checksum [2], [5]. That is necessary but not enough: verification was a manual, optional step that users had to remember, a leaked signing key had no fast recovery path, and nothing limited the damage if verification was skipped. SignGate brings several controls together into one **enforced** gate:

Table 12: Standard fix versus SignGate

Aspect	Standard fix (signed binary + SHA)	SignGate
Verification	Optional, manual, often skipped	Mandatory: the gate is the only way to run the tool
Trust anchor	Key often fetched from same site	Root key pinned in customer repo
Leaked signing key	Re-issue key; every user must update	Root-signed CRL; automatic on next run
Silent key misuse	Undetectable	Transparency log + monitor
Rollback to old version	Not addressed	Signed version + last-seen state
If malicious code runs	All CI secrets exposed	Env allowlist: 1 of 8 secrets exposed
Detection time	About 2 months (2021)	Minutes (monitor interval)

The novel part is **not a new algorithm**. It is the combination of standard, well-analysed primitives into a 'verify-before-execute + least-privilege' pattern that also covers key theft and rollback. It borrows ideas from The Update Framework [18], Certificate Transparency [11] and Sigstore [12], but stays small enough (about 500 lines) for any small team to read and adopt.

8.2 Real-World Relevance

- **Industry alignment:** the design matches SLSA provenance and signing levels [13], the NIST Secure Software Development Framework SP 800-218 [14] and U.S. Executive Order 14028 [15]. EO 14028 was issued in May 2021, partly in response to SolarWinds and Codecov-style supply-chain attacks.
- **Current context:** supply-chain attacks keep recurring. Examples include SolarWinds (2020), 3CX (2023), the xz-utils backdoor (2024) and the tj-actions/changed-files GitHub Action compromise (2025), which again leaked CI secrets. The verify-before-execute pattern applies directly to GitHub Actions, npm/PyPI packages and container images.

- **Cost:** all components are open source. A Cloud KMS key costs about US\$1 per month, and the root HSM can be a low-cost hardware token. The gate adds under 1 ms per build.
- **Feasibility:** it needs only Python and one widely used library. It integrates as a single CI step, and customers change one line in their pipeline.

9. CONCLUSION

The Codecov incident shows how a single leaked storage credential, together with unsigned software and the 'curl | bash' habit, let an attacker steal secrets from thousands of CI pipelines for about two months. The main failure was **integrity and authenticity**: customers had no enforced way to tell the genuine uploader from a modified one. The breach of **confidentiality** followed from that.

SignGate addresses each weakness with standard cryptography. SHA-256 and Ed25519 signatures bind every release to the publisher's key. A root-certified, rotatable, revocable key hierarchy limits the damage of key theft. A hash-chained transparency log and monitor make misuse visible. A fail-closed gate with an environment allowlist ensures nothing unverified runs and exposes very little if something ever does. The prototype blocked the original attack and five stronger variants, accepted the genuine release, reduced exposed secrets from 8 to 1 and cost about 0.6 ms per build.

Future work: keyless signing with Sigstore/OIDC identities, SLSA build provenance to cover a compromised build server, witness co-signing of the transparency log, and packaging SignGate as a reusable GitHub Action.

10. REFERENCES

- [1] Codecov, "Bash Uploader Security Update," Apr. 15, 2021. [Online]. Available: <https://about.codecov.io/security-update/>
- [2] Codecov, "Post-Mortem / Root Cause Analysis (April 2021)," 2021. [Online]. Available: <https://about.codecov.io/apr-2021-post-mortem/>
- [3] J. Menn and R. Satter, "Codecov hackers breached hundreds of restricted customer sites - sources," Reuters, Apr. 19, 2021.
- [4] HashiCorp, "HCSEC-2021-12: Codecov Security Event and HashiCorp GPG Key Exposure," HashiCorp Security Bulletin, Apr. 2021. [Online]. Available: <https://discuss.hashicorp.com>
- [5] Cybersecurity Dive, "Codecov to sunset Bash Uploader following April supply chain attack," 2021. [Online]. Available: <https://www.cybersecuritydive.com/news/codecov-bash-uploader-supply-chain-attack/601757/>
- [6] GitGuardian, "Codecov supply chain attack breakdown," 2021. [Online]. Available: <https://blog.gitguardian.com/codecov-supply-chain-breach/>
- [7] S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," IETF RFC 8032, Jan. 2017.
- [8] NIST, "Secure Hash Standard (SHS)," FIPS PUB 180-4, Aug. 2015.
- [9] NIST, "Digital Signature Standard (DSS)," FIPS PUB 186-5, Feb. 2023.
- [10] E. Barker, "Recommendation for Key Management: Part 1 - General," NIST SP 800-57 Part 1 Rev. 5, May 2020.
- [11] B. Laurie, A. Langley and E. Kasper, "Certificate Transparency," IETF RFC 6962, Jun. 2013.
- [12] Z. Newman, J. S. Meyers and S. Torres-Arias, "Sigstore: Software Signing for Everybody," in Proc. ACM CCS, 2022, pp. 2353-2367.
- [13] OpenSSF, "SLSA: Supply-chain Levels for Software Artifacts, Specification v1.0," 2023. [Online]. Available: <https://slsa.dev>
- [14] M. Souppaya, K. Scarfone and D. Dodson, "Secure Software Development Framework (SSDF) Version 1.1," NIST SP 800-218, Feb. 2022.
- [15] The White House, "Executive Order 14028: Improving the Nation's Cybersecurity," May 12, 2021.
- [16] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," IETF RFC 8446, Aug. 2018.
- [17] A. Shostack, Threat Modeling: Designing for Security. Indianapolis, IN, USA: Wiley, 2014.
- [18] J. Samuel, N. Mathewson, J. Capps and R. Dingleline, "Survivable key compromise in software update systems," in Proc. ACM CCS, 2010, pp. 61-72.
- [19] Python Cryptographic Authority, "pyca/cryptography documentation: Ed25519 signing." [Online]. Available: <https://cryptography.io>